PHASE 5 — DEEP DIVE

Production AI Engineering

LangChain LCEL · LlamaIndex · Vercel AI SDK · Evals · Prompt Versioning · Semantic Caching · Multi-modal · AI UX

Month 14–17 · Every resource with exact video title + channel name

Phase Duration	16 weeks (Month 14 through 17 of the full roadmap)
Daily Commitment	3–4 hours weekdays, 5–6 hours weekends
Your Starting Point	Phase 4 complete — production RAG engine with LangGraph agents live on Railway
Goal of This Phase	Go from 'I can build AI' to 'I can build production AI that earns money' — evals, observability, cost optimisation, model routing, multi-modal, and AI UX that users love
Core Shift	Phase 4 was about building AI systems. Phase 5 is about making those systems reliable, cost-efficient, and user-ready. This is the difference between a demo and a product.
Total Topics	10 topics — LangChain LCEL, LlamaIndex, Vercel AI SDK production, LLM Evals, Prompt versioning, Model routing, Semantic caching, Multi-modal AI, AI UX patterns, Full-stack integration
Capstone Project	AI Research Agent — Portfolio Project #5 (the full-stack showcase combining every skill)

⍰ Phase 5 Philosophy — From Prototype to Production

Phase 4 taught you how to build AI systems that work. Phase 5 teaches you how to build AI systems that work reliably, cheaply, and at scale — with real users depending on them. Every topic in this phase is motivated by a real production problem.

Problem: My AI pipeline breaks when inputs change	Solution (Topic 1): LangChain LCEL chains with typed inputs/outputs and proper error handling
Problem: My RAG is slow for document-heavy queries	Solution (Topic 2): LlamaIndex advanced indexing — hierarchical nodes, query pipelines, router query engines
Problem: Streaming from my Python AI to the browser is messy	Solution (Topic 3): Vercel AI SDK production patterns — useChat, Server Actions, multi-step streaming
Problem: I don't know if my AI got better or worse after changes	Solution (Topic 4): LLM evaluation framework — automated evals, regression detection, eval-driven development
Problem: I accidentally broke my best prompt when editing it	Solution (Topic 5): Prompt versioning with LangSmith — Git for prompts, A/B testing, rollback
Problem: My LLM bill is $2000/month and growing	Solution (Topic 6): Model routing + semantic caching — 60–80% cost reduction with no quality loss
Problem: Users upload images — I can't process them	Solution (Topic 7): Multi-modal AI — vision, audio, document understanding with GPT-4o and Gemini
Problem: My AI UI feels slow and untrustworthy	Solution (Topic 8): AI UX patterns — progressive disclosure, confidence indicators, HITL design
Problem: My Python AI service and Next.js frontend are not integrated properly	Solution (Topic 9): Full-stack AI integration patterns — the production architecture
Problem: I cannot demonstrate cost savings to stakeholders	Solution (Topic 10): AI cost analytics dashboard — track spend, savings, and ROI in real time

TOPIC 1 — LangChain LCEL & Production Chains · *Weeks 1–2 · The composable AI pipeline framework*

LangChain Expression Language (LCEL) is the modern way to build composable, typed AI pipelines. It replaces the old LangChain chains with a clean pipe-operator syntax, built-in streaming, parallel execution, and type safety. Every AI pipeline in this phase uses LCEL.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	LangChain Expression Language (LCEL) — Official Tutorial Series (4 parts)	LangChain (official channel)	YouTube → LangChain official YouTube → Search 'LCEL Tutorial Series LangChain' → open playlist	The official LCEL tutorial from the creators. Watch all 4 parts. Non-negotiable.
2	Video	LangChain LCEL — Full Tutorial with Real Projects	Sam Witteveen	YouTube → Search 'LangChain LCEL Full Tutorial Real Projects Sam Witteveen'	Sam Witteveen is the best practical LangChain YouTuber. Builds real pipelines with LCEL.
3	Video	LangChain LCEL Deep Dive — Runnables, Chains, and Streaming	James Briggs	YouTube → Search 'LangChain LCEL Deep Dive Runnables Chains Streaming James Briggs'	James Briggs explains LCEL internals — Runnable protocol, RunnableParallel, RunnableLambda.
4	Video	LangChain v0.2 — What Changed and How to Migrate	Sam Witteveen	YouTube → Search 'LangChain v0.2 What Changed How to Migrate Sam Witteveen'	LangChain changed significantly in v0.2. Know the current APIs before writing production code.
5	Video	Building Production RAG Chains with LCEL — Best Practices	Lance Martin (LangChain)	YouTube → Search 'Production RAG Chains LCEL Best Practices Lance Martin'	Lance Martin shows exactly how to upgrade your Phase 4 RAG to LCEL-based production chains.
6	Video	LangChain Callbacks and Tracing — Instrument Your AI Pipeline	LangChain (official channel)	YouTube → LangChain YouTube → Search 'LangChain Callbacks Tracing Instrument AI Pipeline'	Callbacks for logging, cost tracking, and LangSmith integration — production observability.
7	Video	Parallel Execution in LangChain LCEL — RunnableParallel	Sam Witteveen	YouTube → Search 'Parallel Execution LangChain LCEL RunnableParallel Sam Witteveen'	Run multiple LLM calls in parallel within one chain — 2–4x faster than sequential.
8	Docs	LangChain Expression Language (LCEL) — Official Documentation	LangChain Team	python.langchain.com/docs/expression_language — read Interface + Primitives + Cookbook	Read the full LCEL docs: Interface, Primitives (RunnableParallel, RunnableLambda), Cookbook.

1.2 — LCEL Core Concepts

The Pipe Operator — The Heart of LCEL

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

# LCEL uses | (pipe) to compose Runnables
# Input flows left → right, each step transforms the state

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert {role}. Answer concisely.'),
    ('human', '{question}'),
])

llm   = ChatOpenAI(model='gpt-4o-mini', temperature=0)
parser = StrOutputParser()

# Chain: prompt | llm | parser
chain = prompt | llm | parser

# Invoke (sync)
result = chain.invoke({'role': 'data analyst', 'question': 'What is RAG?'})

# Stream (async — token by token)
async for chunk in chain.astream({'role': 'data analyst', 'question': 'What is
RAG?'}):
    print(chunk, end='', flush=True)

# Batch (parallel — 10 questions at once)
results = chain.batch([{'role': 'analyst', 'question': q} for q in questions])
```

RunnableParallel — Fan-Out + Fan-In

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# Retrieve context AND rephrase question in PARALLEL (not sequential)
# Then pass both to generation step
retrieval_chain = RunnableParallel({
    'context':  retriever,                  # runs in parallel
    'question': RunnablePassthrough(),      # passes through unchanged
    'history':  history_retriever,          # runs in parallel
})
```

```
# Full RAG chain with parallel retrieval
rag_chain = (
    retrieval_chain
    | generation_prompt
    | llm
    | StrOutputParser()
)

# This runs retriever + history_retriever in parallel threads
# Then passes both results + question to the generation step
# Result: 2x faster than running retriever then history sequentially
```

LCEL for Production — What Changes vs Simple Chains

- Error handling: .with_fallbacks([backup_chain]) — automatic failover if primary chain fails
- Retry: .with_retry(stop_after_attempt=3) — automatic retry with exponential backoff
- Rate limiting: use asyncio.Semaphore wrapping chain.ainvoke() for concurrency control
- Type safety: use Pydantic output parsers — chain.invoke() returns a validated Python object
- Streaming: every LCEL chain supports .astream() natively — no extra code required
- Batching: chain.batch(inputs) processes a list of inputs concurrently — 5–10x throughput boost

> ⍰ The most impactful LCEL pattern for production: add .with_fallbacks([claude_chain]) to your OpenAI chain. When OpenAI returns a 429 or 500, LangChain automatically retries with Claude. Your users never see the error. This takes 5 lines of code.

1.3 — Stop Conditions

☐ Rebuild your Phase 4 RAG pipeline using LCEL — same functionality, cleaner code with | operator
☐ Add RunnableParallel: retrieve from Qdrant AND history from DB simultaneously — measure speed improvement
☐ Add .with_fallbacks(): if OpenAI fails, automatically use Anthropic — test with a mock failure
☐ Add .with_retry(stop_after_attempt=3): chain auto-retries on rate limit errors
☐ Build a branching chain: route to 'simple_chain' or 'advanced_chain' based on query complexity
☐ Stream LCEL chain output to browser via FastAPI StreamingResponse — first token in <1 second
☐ Batch processing: process 100 documents through a summarisation chain — using chain.batch() with concurrency

TOPIC 2 — LlamaIndex: Advanced Document Intelligence · *Weeks 2–3 · Where LangChain ends, LlamaIndex begins*

LlamaIndex is the best framework for document-heavy AI applications. Where LangChain excels at agent orchestration, LlamaIndex excels at document indexing, retrieval, and structured data extraction. It has more sophisticated indexing primitives, better query engines for structured data, and a cleaner abstraction for RAG pipelines over complex document collections.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	LlamaIndex — Full Beginner to Advanced Course	Sam Witteveen	YouTube → Search 'LlamaIndex Full Beginner Advanced Course Sam Witteveen'	Sam's comprehensive LlamaIndex course — best practical tutorial on YouTube.
2	Video	LlamaIndex vs LangChain — When to Use Which	Sam Witteveen	YouTube → Search 'LlamaIndex vs LangChain When to Use Sam Witteveen'	Critical decision video. Watch BEFORE learning LlamaIndex to set the right context.
3	Video	LlamaIndex — Build a Production RAG Application (1.5 hrs)	Rabbitmetrics	YouTube → Search 'LlamaIndex Build Production RAG Application Rabbitmetrics'	End-to-end LlamaIndex RAG — from document loading to query response. Follow along completely.
4	Video	LlamaIndex Router Query Engine — Route Queries to Multiple Indexes	Andrei Dumitrescu (Crystal Mind Academy)	YouTube → Search 'LlamaIndex Router Query Engine Multiple Indexes Andrei Dumitrescu'	Router engine: route 'financial questions' to finance docs, 'legal questions' to legal docs.
5	Video	LlamaIndex Sub-Question Query Engine — Decompose Complex Questions	Sam Witteveen	YouTube → Search 'LlamaIndex Sub-Question Query Engine Decompose Questions Sam Witteveen'	Break 'Compare Q3 2023 vs Q3 2024 earnings' into sub-queries over different documents.
6	Video	LlamaIndex Property Graph Index — Knowledge Graph RAG	LlamaIndex (official channel)	YouTube → LlamaIndex official YouTube → Search 'Property Graph Index Knowledge Graph RAG'	Graph-based RAG for complex relational knowledge — entities + relationships + communities.
7	Video	LlamaIndex Workflows — Event-Driven Agent Orchestration	LlamaIndex (official channel)	YouTube → LlamaIndex official YouTube → Search 'Workflows Event-Driven Agent Orchestration'	LlamaIndex Workflows: a cleaner agent framework than LangGraph for document pipelines.
8	Video	LlamaIndex Query Pipelines — Build Complex RAG Pipelines Declaratively	Sam Witteveen	YouTube → Search 'LlamaIndex Query Pipelines Complex RAG Declaratively Sam Witteveen'	Declarative pipeline syntax — define RAG pipeline as a graph, visualise it, optimise it.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Video	LlamaIndex Structured Data — Query SQL, CSV, and JSON with Natural Language	Andrei Dumitrescu (Crystal Mind Academy)	YouTube → Search 'LlamaIndex Structured Data Query SQL CSV JSON Natural Language Andrei'	Query your PostgreSQL database with natural language — NL2SQL with LlamaIndex.
10	Docs	LlamaIndex Official Documentation — Core Concepts	LlamaIndex Team	docs.llamaindex.ai — read Getting Started + Understanding (all modules)	Good docs. Read Getting Started + Understanding section before writing any code.

2.2 — What to Study

LlamaIndex vs LangChain — When to Use Each

Use Case	LangChain (LCEL + LangGraph)	LlamaIndex
Agent orchestration	✅Excellent — LangGraph is purpose-built for complex agent workflows	⚠ LlamaIndex Workflows works but less mature
Multi-step RAG pipelines	✅LCEL chains are clean and composable	✅Query Pipelines are declarative and visual
Document indexing	⚠ Basic chunking + vector store, limited indexing types	✅Hierarchical, property graph, summary indexes — rich options
Structured data queries	⚠ SQL chain exists but limited	✅NL2SQL, structured data QA — first-class support
Multi-document reasoning	⚠ Works but requires manual orchestration	✅Sub-question engine decomposes across document collections
Knowledge graphs	⚠ Limited native support	✅Property Graph Index — native KG-RAG support
Production observability	✅LangSmith — the best LLM observability tool	✅LlamaTrace (built on Arize Phoenix)

LlamaIndex Core Abstractions

- Document: raw content + metadata — loaded by Document Loaders
- Node: a chunk of a document with relationships to parent/child nodes
- Index: a data structure for efficient retrieval — VectorStoreIndex, SummaryIndex, PropertyGraphIndex
- Retriever: extracts relevant nodes from an index given a query
- Query Engine: combines retriever + response synthesiser — the end-to-end RAG interface
- Router Query Engine: routes queries to different sub-engines based on description

Property Graph Index — The Most Powerful LlamaIndex Feature

- Extracts entities (Company, Person, Product) and relationships (ACQUIRED, FOUNDED, WORKS_AT) from documents
- Stores in a graph database (Neo4j or in-memory)
- Enables multi-hop reasoning: 'Who founded the company that acquired Figma?' — requires 2 hops

- Combines vector search + graph traversal for maximum retrieval quality

Use LlamaIndex Property Graph Index for any domain with complex relationships: legal (cases, precedents, judges), financial (companies, executives, transactions), medical (patients, conditions, treatments). It enables questions that pure vector search cannot answer.

2.3 — Stop Conditions

- ☐ Build a LlamaIndex VectorStoreIndex over a 100-document corpus — compare retrieval quality vs Phase 4 Qdrant approach
- ☐ Implement Router Query Engine: route finance questions to finance index, legal to legal index — correct routing on 20 test queries
- ☐ Implement Sub-Question Query Engine: complex multi-hop question correctly decomposed into 3 sub-queries
- ☐ Build NL2SQL: user asks 'show top 10 users by token usage last month' — correct SQL generated and executed
- ☐ Build Property Graph Index over a company knowledge base — multi-hop queries correctly answered
- ☐ Compare LlamaIndex vs LangChain LCEL on same RAG task — document the quality and speed differences

TOPIC 3 — Vercel AI SDK: Production Streaming Patterns · *Week 3 · The bridge between Python AI and React UI*

You learned the Vercel AI SDK basics in Phase 1. Phase 5 goes deeper — multi-step tool streaming, generative UI, Server Actions with streaming, AI state management, and connecting your Python FastAPI backend to the Vercel AI SDK. This is where your full-stack AI product comes together.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Vercel AI SDK 3.4 — What's New (Generative UI, Multi-Step Tools, AI State)	Vercel (official channel)	YouTube → Vercel official YouTube → Search 'AI SDK 3.4 New Generative UI Multi-Step Tools'	Official overview of AI SDK 3.x features. Watch to understand what's available before diving in.
2	Video	Generative UI with Vercel AI SDK — RSC and useUIState	Delba de Oliveira (Vercel)	YouTube → Search 'Generative UI Vercel AI SDK RSC useUIState Delba'	Generative UI: LLM streams React components, not just text. Mind-bending UX capability.
3	Video	Multi-Step Tool Calling with Vercel AI SDK — Build a Research Agent	Vercel (official channel)	YouTube → Vercel YouTube → Search 'Multi-Step Tool Calling Vercel AI SDK Research Agent'	AI SDK handles multi-step tool calls natively — no manual loop. Essential for agentic UIs.
4	Video	useChat Hook Deep Dive — Streaming, Optimistic Updates, Error Handling	Delba de Oliveira (Vercel)	YouTube → Search 'useChat Hook Deep Dive Streaming Optimistic Delba Vercel'	The most complete useChat tutorial — covers options, handlers, and edge cases.
5	Video	Connecting a Custom Backend to Vercel AI SDK	Jack Herrington	YouTube → Search 'Connecting Custom Backend Vercel AI SDK Jack Herrington'	How to use AI SDK useChat with your own FastAPI backend — not just Next.js route handlers.
6	Video	Server Actions + Vercel AI SDK — The Full Production Pattern	Josh tried coding	YouTube → Search 'Server Actions Vercel AI SDK Full Production Pattern Josh tried coding'	Combine Next.js 14 Server Actions with AI SDK streaming — the cleanest full-stack AI pattern.
7	Video	Vercel AI SDK with LangChain — Stream from Python to React	Sam Witteveen	YouTube → Search 'Vercel AI SDK LangChain Stream Python React Sam Witteveen'	Connect your Python LangChain/LangGraph backend to Vercel AI SDK frontend — key integration.
8	Docs	Vercel AI SDK Documentation — Guides section	Vercel Team	sdk.vercel.ai/docs/guides — read Chatbot, Multi-Step Tools, Generative UI, Custom Backends	The AI SDK docs are excellent and concise. Read the full Guides section — under 2 hours.

3.2 — Production Patterns

Pattern 1: useChat with Custom Backend (FastAPI + LangGraph)

```
// Next.js — useChat pointing at your FastAPI AI service
import { useChat } from 'ai/react';

export function ChatInterface() {
  const { messages, input, handleInputChange, handleSubmit, isLoading } =
useChat({
    // Point to your Fastify API (which proxies to FastAPI)
    api: '/api/v1/chat',

    // Optimistic updates — user message appears instantly
    optimisticUpdate: true,

    // Handle tool call results in the UI
    onToolCall: async ({ toolCall }) => {
      if (toolCall.toolName === 'showDocument') {
        return <DocumentViewer id={toolCall.args.docId} />;
      }
    },

    // Error handling — show toast, not broken UI
    onError: (error) => toast.error('AI service temporarily unavailable'),
  });

  return (/* your chat UI */);
}

// Fastify route — proxies streaming response from FastAPI
// fastify.post('/api/v1/chat', async (req, reply) => {
//   const stream = await fetch('http://fastapi:8000/chat', {
//     method: 'POST', body: JSON.stringify(req.body),
//     headers: { 'Content-Type': 'application/json' }
//   });
//   return reply.send(stream.body);  // pipe the stream through
// });
```

Pattern 2: Generative UI — LLM Streams React Components

- Traditional AI: LLM streams text → frontend renders text
- Generative UI: LLM decides to render a Chart, a Table, a Map, a ProductCard
- Use case: 'Show me sales trends' → LLM renders a LineChart component with real data
- Use case: 'Find me flights to Tokyo' → LLM renders FlightCard components
- Implementation: createStreamableUI() in Server Actions + useUIState() in client

Generative UI is the single most impressive AI UX capability in your toolkit. When a user asks a data question and a chart instantly appears instead of text — that converts users. Add one Generative UI feature to your capstone project.

3.3 — Stop Conditions

- ☐ useChat connected to your FastAPI/LangGraph backend — full streaming working, not just OpenAI route
- ☐ Multi-step tool calling: agent uses search tool, then calculator, then writes final answer — all streamed to UI
- ☐ Generative UI: one feature where LLM returns a React component (chart, table, or card) instead of text
- ☐ Optimistic updates: user message appears instantly without waiting for AI response
- ☐ Error recovery: when AI backend returns 503, UI shows retry button — not a blank screen
- ☐ Token usage display: show live token count and estimated cost below each AI response

TOPIC 4 — LLM Evaluation: Automated, Continuous, Production-Grade · *Weeks 4–5 · Evals are the CI/CD of AI engineering*

You cannot ship reliable AI without evaluation. Every time you change a prompt, add a document to your RAG index, update an embedding model, or modify a retrieval strategy — you need to know if your system got better or worse. Automated LLM evaluation is the single most underrated skill that separates junior AI engineers from senior ones.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	LLM Evaluation — Full Course (DeepLearning.AI, free)	DeepLearning.AI	learn.deeplearning.ai → Search 'Evaluating and Debugging Generative AI' — free	Andrew Ng's platform + W&B team. The definitive free course on LLM evaluation. Watch completely.
2	Video	RAGAS — Complete Guide to RAG Evaluation (RAGAS v0.2)	Sam Witteveen	YouTube → Search 'RAGAS Complete Guide RAG Evaluation v0.2 Sam Witteveen'	Most up-to-date RAGAS tutorial. Covers all metrics, test set generation, and CI integration.
3	Video	LLM-as-Judge — Using GPT-4o to Evaluate Other LLMs	Sam Witteveen	YouTube → Search 'LLM as Judge Using GPT-4o Evaluate Other LLMs Sam Witteveen'	LLM-as-judge is the most scalable evaluation method. Sam shows how to implement it correctly.
4	Video	LangSmith Evaluation — Create Datasets, Run Experiments, Compare	LangChain (official channel)	YouTube → LangChain YouTube → Search 'LangSmith Evaluation Datasets Experiments Compare'	Full LangSmith eval workflow. Create dataset → run evaluator → compare experiments.
5	Video	Evaluation-Driven Development for LLMs — How to Ship with Confidence	Hamel Husain	YouTube → Search 'Evaluation Driven Development LLMs Ship Confidence Hamel Husain'	Hamel is one of the top AI evaluation experts. This talk changes how you think about evals.
6	Video	Braintrust — Modern LLM Evaluation Platform (Full Tutorial)	Braintrust (official channel)	YouTube → Braintrust official YouTube → Search 'Braintrust LLM Evaluation Platform Tutorial'	Braintrust is an excellent LangSmith alternative with a better dataset management UI.
7	Video	Automated Test Generation for LLM Applications — Synthetic Data	James Briggs	YouTube → Search 'Automated Test Generation LLM Applications Synthetic Data James Briggs'	Use GPT-4o to generate eval datasets — the fastest way to get a 100-question test set.
8	Video	Continuous Evaluation — Run LLM Evals in GitHub Actions CI/CD	Hamel Husain	YouTube → Search 'Continuous Evaluation LLM Evals GitHub Actions CI/CD Hamel Husain'	Gate every code merge on eval scores — evals in CI is what makes AI engineering rigorous.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
9	Blog	Your AI Product Needs Evals (by Hamel Husain + Eugene Yan — essential reading)	Hamel Husain + Eugene Yan	hamel.dev → Search 'Your AI Product Needs Evals'	The most important blog post on LLM evaluation. Read before writing a single eval test.

4.2 — The Evaluation Framework

Level 1: Unit Tests (Fast, Cheap, Deterministic)

- Test exact output format: response is valid JSON matching your Pydantic schema
- Test output constraints: response length is within bounds, no profanity, no PII
- Test tool calling: given query X, tool Y is called with arguments Z
- Test retrieval: given query X, document D is in the top-K results
- Implementation: pytest + mock LLM responses — runs in <10 seconds

Level 2: LLM-as-Judge (Scalable, Semi-Automated)

- Concept: use GPT-4o to grade the output of your smaller/cheaper models
- Criteria: correctness, helpfulness, safety, format adherence — defined in a rubric prompt
- Output: score (1–5) + justification — both stored for human review
- Scale: evaluate 1000 responses in minutes for $2 — not feasible with humans
- Bias risk: LLM judges are biased toward length, formality, and their own outputs

```
# LLM-as-Judge pattern
JUDGE_PROMPT = """
You are evaluating an AI assistant response to a user question.

Question: {question}
Reference Answer: {reference}
AI Response: {response}

Rate the AI response on a scale of 1-5 for EACH criterion:
1. Correctness: Is the answer factually accurate vs the reference?
2. Completeness: Does the answer cover all key points?
3. Conciseness: Is the answer appropriately brief without missing info?

Respond in JSON: {{'correctness': int, 'completeness': int,
                   'conciseness': int, 'overall': int,
                   'justification': str}}
"""

async def judge_response(question, reference, response):
    result = await judge_client.chat.completions.create(
        model='gpt-4o',           # use the best model as judge
        response_format={'type': 'json_object'},
        messages=[{'role': 'user',
```

```
                    'content': JUDGE_PROMPT.format(...)}]
    )
    return json.loads(result.choices[0].message.content)
```

Level 3: RAGAS — Automated RAG-Specific Metrics

- Faithfulness: does the answer cite only information from retrieved context? (prevents hallucination)
- Answer Relevancy: is the answer relevant to the question? (prevents off-topic responses)
- Context Precision: are retrieved chunks relevant? (measures retrieval quality)
- Context Recall: were all necessary chunks retrieved? (measures retrieval completeness)
- Answer Correctness: is the answer factually correct vs reference? (requires golden dataset)

Level 4: Continuous Evaluation in CI/CD

```
# .github/workflows/eval.yml — run evals on every PR
name: LLM Evaluation Gate
on: [pull_request]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run RAGAS evaluation
        run: |
          python scripts/run_evals.py \
            --dataset golden_test_set.json \
            --thresholds faithfulness=0.85,answer_relevancy=0.80

      - name: Check thresholds
        run: |
          if [ $FAITHFULNESS < 0.85 ]; then
            echo 'FAIL: Faithfulness regression detected'
            exit 1
          fi

# Result: PRs that reduce eval quality are BLOCKED from merging
# This is eval-driven development — the gold standard for AI engineering
```

The golden test set is sacred. Never put examples from it into training or prompt examples. Keep it isolated. Add 10 new questions per week as you discover edge cases. A 200-question golden test set that never leaks is worth more than 2000 examples that do.

4.3 — Stop Conditions

- ☐ Golden test set: 100 question-answer pairs covering all query types your RAG handles

☐ Level 1 evals: pytest suite testing JSON schema, length constraints, and tool call selection
☐ Level 2 evals: LLM-as-judge scoring 100 responses — results stored with justifications in PostgreSQL
☐ Level 3 evals: RAGAS scores for all 5 metrics on golden test set — all above thresholds
☐ Level 4 evals: GitHub Actions CI blocks merges that reduce faithfulness below 0.85
☐ Eval dashboard: weekly chart of all 5 RAGAS metrics over time — visible regressions caught
☐ Synthetic test generation: used GPT-4o to generate 50 additional test cases — validated by hand

TOPIC 5 — Prompt Versioning, A/B Testing & Management · *Week 5 · Git for prompts — because prompts are code*

Prompts are the most fragile part of any AI system. A single word change can dramatically improve or destroy quality. Without version control, you cannot roll back a bad change, compare variants, or audit what changed when quality dropped. Prompt versioning is non-negotiable in production.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	LangSmith Prompt Hub — Version Control for Prompts	LangChain (official channel)	YouTube → LangChain YouTube → Search 'LangSmith Prompt Hub Version Control Prompts'	Official LangSmith Prompt Hub tutorial. Push/pull prompts like Git commits.
2	Video	Prompt Management at Scale — How Production Teams Handle Prompts	Hamel Husain	YouTube → Search 'Prompt Management Scale Production Teams Hamel Husain'	Hamel's talk on how real AI engineering teams manage prompts in production.
3	Video	A/B Testing LLM Prompts — How to Run Prompt Experiments	Sam Witteveen	YouTube → Search 'AB Testing LLM Prompts Run Prompt Experiments Sam Witteveen'	A/B test prompt variants on a fixed dataset — data-driven prompt iteration.
4	Video	Prompt Optimization with DSPy — Automatic Prompt Engineering	Connor Shorten	YouTube → Search 'Prompt Optimization DSPy Automatic Prompt Engineering Connor Shorten'	DSPy automatically optimises prompts using your eval dataset — the future of prompt engineering.
5	Video	PromptLayer Tutorial — Prompt Versioning and Analytics	PromptLayer (official channel)	YouTube → PromptLayer YouTube → Search 'PromptLayer Tutorial Versioning Analytics'	PromptLayer is a simpler prompt management alternative to LangSmith Prompt Hub.
6	Blog	Why Prompt Engineering Is More Like Software Engineering Than You Think	Anthropic Research Blog	anthropic.com → Research blog → Search 'Prompt Engineering Software Engineering'	Anthropic's perspective on treating prompts as first-class engineering artifacts.

5.2 — Prompt Versioning System Design

The Minimum Viable Prompt Versioning System

```
# Option A: Store prompts in your database (simple, works well)
# Table: prompts
# (id, name, version, template, metadata, created_at, is_active)
```

```
# Option B: LangSmith Prompt Hub (recommended for teams)
from langsmith import Client

client = Client()

# Pull a specific prompt version
prompt = client.pull_prompt('rag-synthesis:v12')  # version pinned

# Push a new version after testing
client.push_prompt('rag-synthesis:v13', object=new_prompt)

# Promote to production after A/B test confirms improvement
client.push_prompt('rag-synthesis:latest', object=new_prompt)

# In production code: always pin the version
# NEVER use 'latest' in production — you'll get unexpected changes
SYNTHESIS_PROMPT_VERSION = 'rag-synthesis:v12'  # in config.py

# A/B test: 50% of traffic gets v12, 50% gets v13
# After 1000 queries: compare eval scores → promote the winner
```

DSPy — Automatic Prompt Optimisation (Advanced)

- DSPy treats prompt engineering as a machine learning problem
- You define the task (input → output) and the evaluation metric
- DSPy automatically finds the best prompt for your task using your eval dataset
- BootstrapFewShot: DSPy finds the best few-shot examples automatically
- MIPROv2: the most powerful DSPy optimiser — optimises both instructions and examples

> ⍰ DSPy is overkill for most tasks but invaluable when you have a difficult structured extraction task and a good eval set. Let DSPy run overnight, and wake up to a 15–30% better prompt than you wrote by hand. It is the closest thing to automated prompt engineering.

5.3 — Stop Conditions

☐ All production prompts stored in LangSmith Prompt Hub with version numbers — no prompts hardcoded in code
☐ Rollback tested: deploy v12 → check metrics → rollback to v11 in <60 seconds
☐ A/B test: compare two prompt versions on golden test set — winner determined with statistical significance
☐ DSPy: run BootstrapFewShot on one prompt — compare DSPy-optimised vs hand-written quality
☐ Prompt diff: when a prompt changes, the diff is reviewed in code review just like code changes

TOPIC 6 — Model Routing & Semantic Caching · *Week 6 · Cut your LLM bill by 60–80% with no quality loss*

LLM API costs are the biggest expense in any AI SaaS. Model routing sends simple queries to cheap models and complex queries to expensive ones. Semantic caching returns cached responses for semantically similar queries — not just identical ones. Together, these two techniques routinely cut 60–80% of LLM costs for production AI applications.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	LLM Router — How to Route Queries to the Right Model Automatically	Sam Witteveen	YouTube → Search 'LLM Router Route Queries Right Model Automatically Sam Witteveen'	Build a classifier that routes queries — fast and cheap.
2	Video	RouteLLM — Open Source LLM Router (Routing with Preference Data)	Anyscale (official channel)	YouTube → Anyscale YouTube → Search 'RouteLLM Open Source LLM Router Preference Data'	RouteLLM from Anyscale — open source router trained on human preference data. Use in production.
3	Video	Semantic Caching for LLM Applications — GPTCache Tutorial	Zep AI (official channel)	YouTube → Search 'Semantic Caching LLM Applications GPTCache Tutorial Zep AI'	GPTCache is the most popular semantic caching library. Full tutorial with Redis backend.
4	Video	LLM Caching with LangChain — InMemory, SQLite, Redis, Semantic Cache	Sam Witteveen	YouTube → Search 'LLM Caching LangChain InMemory SQLite Redis Semantic Cache Sam Witteveen'	All LangChain caching options including semantic cache with embeddings.
5	Video	Reducing OpenAI API Costs — 10 Techniques That Actually Work	AI Explained	YouTube → Search 'Reducing OpenAI API Costs 10 Techniques AI Explained'	Comprehensive cost reduction guide — covers routing, caching, batching, and prompt compression.
6	Video	Prompt Compression — LLMLingua — Reduce Token Count by 3–20x	James Briggs	YouTube → Search 'Prompt Compression LLMLingua Reduce Token Count James Briggs'	LLMLingua removes unimportant tokens from prompts — 3–20x token reduction with minimal quality loss.
7	Video	OpenRouter — Single API for 200+ LLM Models with Automatic Fallback	David Ondrej	YouTube → Search 'OpenRouter Single API 200 LLM Models Automatic Fallback David Ondrej'	OpenRouter gives you access to every model through one API — simplifies multi-model routing.

6.2 — Model Routing Architecture

4-Tier Model Routing Strategy

Tier 0 — Cache Hit	Return exact or semantic cache result — $0 cost, <5ms latency. Target: 30–50% of requests
Tier 1 — Micro Model	GPT-4o-mini / Claude 3 Haiku — classification, simple Q&A, summarisation. ~$0.0002/request
Tier 2 — Standard Model	GPT-4o / Claude 3.5 Sonnet — complex reasoning, multi-step tasks. ~$0.003/request
Tier 3 — Power Model	o1 / Claude 3 Opus — maximum quality for critical enterprise queries. ~$0.015/request

Routing Classifier Implementation

```
# Query complexity classifier — routes to the right tier
# Train a small classifier on 500 examples labelled by difficulty
# Inference: <5ms, negligible cost

ROUTING_PROMPT = """
Classify this query by complexity. Respond with ONLY one word.

SIMPLE: factual lookup, single-step, no reasoning required
STANDARD: multi-step reasoning, synthesis, moderate complexity
COMPLEX: deep analysis, multi-document reasoning, critical tasks

Query: {query}
Classification:"""

async def route_query(query: str, context: dict) -> str:
    # Fast path: check cache first
    if cached := await semantic_cache.get(query):
        return cached  # $0 cost

    # Classify query complexity using cheapest model
    tier = await classify_complexity(query)  # GPT-4o-mini, ~$0.0001

    # Route to appropriate model
    match tier:
        case 'SIMPLE':   model = 'gpt-4o-mini'      # $0.15/M tokens
        case 'STANDARD': model = 'gpt-4o'            # $2.50/M tokens
        case 'COMPLEX':  model = 'o1'                # $15.00/M tokens

    response = await complete(query, model=model)
    await semantic_cache.set(query, response)  # cache for next time
    return response
```

Semantic Caching Implementation

```
# Semantic cache: cache by meaning, not exact string
```

```
# 'What is the capital of France?' and 'France's capital city?' → same cache hit

import numpy as np
from redis import Redis

class SemanticCache:
    def __init__(self, threshold=0.95, ttl=86400):  # 24h TTL
        self.threshold = threshold   # cosine similarity threshold
        self.redis = Redis()
        self.embedder = OpenAIEmbeddings(model='text-embedding-3-small')

    async def get(self, query: str) -> str | None:
        query_emb = await self.embedder.aembed_query(query)

        # Check against stored query embeddings
        for key in self.redis.scan_iter('cache:*'):
            stored = json.loads(self.redis.get(key))
            similarity = cosine_similarity(query_emb, stored['embedding'])
            if similarity >= self.threshold:
                return stored['response']  # cache HIT

        return None  # cache MISS

    async def set(self, query: str, response: str):
        embedding = await self.embedder.aembed_query(query)
        self.redis.setex(
            f'cache:{hashlib.sha256(query.encode()).hexdigest()}',
            self.ttl,
            json.dumps({'embedding': embedding, 'response': response})
        )

# In production: use Qdrant as the cache backend for better performance
# Qdrant's vector search is O(log n) vs Redis linear scan
```

6.3 — Stop Conditions

- ☐ Model router: 20 test queries routed correctly to SIMPLE/STANDARD/COMPLEX — verified manually
- ☐ Semantic cache: '95% similar' queries return cached response — demonstrate with 10 paraphrase pairs
- ☐ Cost measurement: log actual cost per request — weekly cost dashboard shows tier distribution
- ☐ LLMLingua: compress a 2000-token RAG context to 500 tokens — measure quality loss on eval set
- ☐ OpenRouter integration: failover to alternative model when primary returns 429 — fully transparent to user
- ☐ ROI calculation: measure monthly cost before vs after routing + caching — document the savings

TOPIC 7 — Multi-Modal AI: Vision, Audio & Documents · *Weeks 7–8 · Text was just the beginning*

The most valuable AI products in 2025 are multi-modal. Users want to upload an invoice photo and have it extracted, describe a UI and have it built, record a voice note and have it summarised. GPT-4o and Gemini 1.5 are natively multi-modal. This topic teaches you to handle every input modality that users actually send.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	GPT-4o Vision — Complete Tutorial (Image Understanding + Structured Extraction)	Dave Gray	YouTube → Search 'GPT-4o Vision Complete Tutorial Image Understanding Dave Gray'	Comprehensive GPT-4o vision tutorial — image description, OCR, structured extraction from photos.
2	Video	OpenAI Whisper — Speech to Text Complete Guide	Tech With Tim	YouTube → Search 'OpenAI Whisper Speech to Text Complete Guide Tech With Tim'	Whisper is the best open-source speech-to-text model. Covers API + local inference.
3	Video	OpenAI Text-to-Speech (TTS) — Build Voice AI Applications	Dave Gray	YouTube → Search 'OpenAI Text to Speech TTS Build Voice AI Applications Dave Gray'	TTS for AI voice responses. Covers streaming audio for low-latency voice applications.
4	Video	Gemini 1.5 Pro — Process Video, Audio, and PDF Files (Complete Tutorial)	Patrick Loeber (Python Engineer)	YouTube → Search 'Gemini 1.5 Pro Process Video Audio PDF Files Patrick Loeber'	Gemini's 1M context window + multi-modal input. Upload a PDF and ask questions. Cover audio analysis.
5	Video	Build a Voice AI Assistant — Speech-to-Text + LLM + Text-to-Speech	Pixegami	YouTube → Search 'Build Voice AI Assistant Speech Text LLM Text Speech Pixegami'	End-to-end voice assistant: Whisper → GPT-4o → TTS. Follow along completely.
6	Video	Document AI — Extract Data from PDFs and Images with GPT-4o Vision	James Briggs	YouTube → Search 'Document AI Extract Data PDFs Images GPT-4o Vision James Briggs'	Multi-modal document extraction — combine OCR + vision for maximum accuracy on complex documents.
7	Video	ElevenLabs Voice Cloning — Custom Voice in Your AI Application	Mervin Praison	YouTube → Search 'ElevenLabs Voice Cloning Custom Voice AI Application Mervin Praison'	Custom AI voice for branded voice assistants — clone a voice from audio samples.
8	Video	DALL-E 3 Image Generation — Complete API Tutorial	Dave Gray	YouTube → Search 'DALL-E 3 Image Generation Complete API Tutorial Dave Gray'	AI image generation for product features — generate product visuals, UI mockups, social content.
9	Video	Multi-Modal RAG — Process Images, Charts, and Tables in Documents	James Briggs	YouTube → Search 'Multi-Modal RAG Process Images Charts	Embed images alongside text in your RAG pipeline

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
				Tables Documents James Briggs'	— retrieve based on image content.
10	Video	Real-Time Voice AI with WebSockets — Build a Live Voice Assistant	Fireship	YouTube → Search 'Real-Time Voice AI WebSockets Live Voice Assistant Fireship'	Real-time streaming audio: user speaks → transcript → LLM → TTS → plays audio. <500ms latency.

7.2 — What to Build

Vision: Document + Image Understanding

- Invoice photo extraction: user photographs an invoice → GPT-4o extracts all fields as JSON
- Chart/graph understanding: user uploads a chart image → LLM describes trends and data
- UI screenshot analysis: 'What's wrong with this UI?' → GPT-4o gives specific feedback
- Multi-page PDF as image: send each PDF page as an image → handles complex layouts that text extraction misses
- Multi-modal RAG: index document images (not just text) — retrieve by visual similarity

Audio: Voice AI Integration

- Voice transcription: user records voice note → Whisper API transcribes to text → sent to your AI
- Voice response: AI text response → OpenAI TTS → streamed audio plays in browser
- Real-time voice: WebSocket → Whisper streaming → LLM streaming → TTS streaming — <500ms latency
- Meeting transcription: upload meeting recording → transcribe → summarise → extract action items

Multi-Modal RAG — The Advanced Pattern

```
# Multi-modal RAG: index text + images together
# Technique: ColPali or GPT-4o Vision for image embedding

# Option A: GPT-4o Vision to describe images → embed description
async def index_document_page(page_image: bytes) -> None:
    # Step 1: Use GPT-4o Vision to describe the page
    description = await gpt4o.chat.completions.create(
        model='gpt-4o',
        messages=[{'role': 'user', 'content': [
            {'type': 'image_url', 'image_url': {'url':
f'{b64}'}},
            {'type': 'text', 'text': 'Describe all text, charts, tables on this
page in detail'},
        ]}]
    ).choices[0].message.content

    # Step 2: Embed the description (not the image itself)
    embedding = await embed(description)
```

```
    # Step 3: Store in Qdrant with original image as payload
    await qdrant.upsert(collection='documents', points=[{
        'id': page_id,
        'vector': embedding,
        'payload': {'description': description, 'image_b64': b64, 'page':
page_num}
    }])
```

Multi-modal RAG is the single most impressive feature you can demo. When a user uploads a 50-page PDF with charts and tables, and your system correctly answers 'What was the Q3 2024 revenue trend?' by locating the right chart — that is a wow moment. Add this to your capstone.

7.3 — Stop Conditions

- ☐ Vision extraction: user uploads invoice photo → all fields (vendor, amount, date, items) extracted as JSON
- ☐ Whisper transcription: 5-minute audio file transcribed in <30 seconds via API
- ☐ Voice assistant: speak a question → LLM answers → TTS plays response — end-to-end in <3 seconds
- ☐ Multi-modal RAG: PDF with charts indexed — query about chart data returns correct answer
- ☐ Document comparison: two contract PDF images — GPT-4o identifies the key differences
- ☐ Image generation: DALL-E 3 integrated — users can generate images from text in your AI app

TOPIC 8 — AI UX Design Patterns · *Week 8–9 · The difference between a tool users tolerate and one they love*

AI applications have unique UX challenges that normal web app patterns don't address: latency (AI is slow), uncertainty (AI is sometimes wrong), opacity (AI decisions are hard to explain), and trust (users don't know if they can rely on AI answers). This topic covers the patterns that the best AI products use to solve these problems.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	AI UX Design Patterns — How to Design AI Products That Users Trust	Lenny's Podcast (YouTube)	YouTube → Search 'AI UX Design Patterns Design AI Products Users Trust Lenny Podcast'	The best product-thinking video on AI UX. Covers trust, uncertainty, and progressive disclosure.
2	Video	Streaming UI — How to Build Great Streaming Experiences in React	Vercel (official channel)	YouTube → Vercel YouTube → Search 'Streaming UI Build Great Streaming Experiences React'	The UX of streaming: skeleton screens, progressive loading, abort handlers.
3	Video	Building Accessible AI Interfaces — ARIA, Focus Management, Screen Readers	Kevin Powell	YouTube → Search 'Building Accessible AI Interfaces ARIA Focus Management Kevin Powell'	AI chat interfaces must be accessible. Screen reader support for streaming text updates.
4	Video	Human-in-the-Loop UX — Designing AI Approval Workflows	Design+AI Conference (YouTube)	YouTube → Search 'Human-in-the-Loop UX Designing AI Approval Workflows Design AI Conference'	How to design HITL flows that feel natural — not like an error state.
5	Video	Framer Motion — Animate Your AI Streaming UI	Fireship	YouTube → Search 'Framer Motion Animate Streaming UI Fireship'	Smooth animations for AI responses — token fade-in, skeleton loaders, transition effects.
6	Video	Claude.ai UX Analysis — What Makes It the Best AI Chat Interface	Theo (t3.gg)	YouTube → Search 'Claude.ai UX Analysis Best AI Chat Interface Theo t3gg'	Deconstruct the UX choices in the best AI chat interfaces — copy what works.
7	Blog	Designing for AI — A Pattern Library for AI UX (NN/g report summary)	Nielsen Norman Group	nngroup.com → Search 'Designing for AI Pattern Library'	The definitive AI UX research from NN/g. Read the free summary article — full report is paid.

8.2 — The 10 AI UX Patterns Every Engineer Must Know

Pattern 1: Progressive Disclosure

- Show a short answer first, then offer 'See more detail' expansion
- Why: AI responses are often verbose. Users want the answer first, details on demand

- Implementation: generate a 1-sentence summary first, then stream the full response below

Pattern 2: Confidence Indicators

- Show the AI's confidence level alongside its answer (High/Medium/Low or a percentage)
- Implementation: ask the LLM to self-assess confidence in its answer (0–100)
- Why: users trust AI more when it accurately represents its own uncertainty

Pattern 3: Source Citations

- Always show which documents were used to generate a RAG answer
- Click source → highlight the relevant passage in the original document
- Why: citations allow users to verify answers — the #1 trust builder for AI responses

Pattern 4: Graceful Streaming

- Show a typing cursor during generation — users know something is happening
- Show 'Thinking...' with animated dots before first token arrives
- Provide an Abort button — users must be able to cancel a slow response
- On error: show friendly message + retry button, never a blank screen or error stack trace

Pattern 5: Action Confirmation (HITL UX)

- Before any irreversible action (send email, delete file, make payment): show a preview
- 'The agent will send this email to john@example.com. Confirm?' with Accept/Edit/Cancel
- After action: show what was done with an Undo option (where possible)

Pattern 6: Suggested Follow-Ups

- After every AI response, show 3 suggested follow-up questions
- Generate follow-ups from the conversation context using a cheap model (GPT-4o-mini)
- Why: reduces cognitive load — users often don't know what to ask next

Pattern 7: Feedback Loops

- Thumbs up/down on every response — sends signal to your eval system
- 'Correct this answer' option — user edits the response, logged as a training example
- Regenerate with different approach — model tries again with varied prompt

Pattern 8: Transparent Reasoning

- Show the agent's thinking process in a collapsible 'Thinking...' section
- List the steps the agent took: 'Searched 3 documents, found 2 relevant chunks, synthesised answer'
- Why: users trust AI more when they can see HOW it arrived at the answer

Pattern 9: Context Awareness Display

- Show the user what context the AI is working with: 'Using 3 of your documents: Contract.pdf, Q3.pdf'
- Token usage indicator: 'Using 2,847 / 128,000 tokens' — prevents context overflow surprise

Pattern 10: Skeleton Screens (Not Spinners)

- Never show a spinner while waiting for AI — show a skeleton of the expected output
- Example: if AI will return a table, show a skeleton table with animated shimmer
- Why: skeleton screens feel 40% faster than spinners — Google's research confirms this

Pick your 3 most impactful AI UX patterns and implement them in your capstone project: (1) Source citations, (2) Streaming with abort, (3) Suggested follow-ups. These three patterns alone will make your AI application feel more professional than 90% of demos.

8.3 — Stop Conditions

☐ Source citations: every RAG response shows clickable source links — clicking highlights passage
☐ Streaming UX: typing cursor, 'Thinking...' state, abort button — all working correctly
☐ Suggested follow-ups: 3 relevant follow-up questions generated after every response
☐ HITL confirmation: agent email-sending feature shows preview before sending — Accept/Edit/Cancel
☐ Confidence indicator: high/medium/low shown alongside AI responses based on context quality
☐ Accessibility: AI chat interface scores 100 on Lighthouse accessibility — keyboard navigable, ARIA labels
☐ Error recovery: network error shows retry button; AI error shows friendly message — no blank screens

TOPIC 9 — Full-Stack AI Integration Architecture · *Week 9–10 · Connecting all four phases into one product*

By now you have four separate systems: Next.js frontend (Phase 1), Fastify backend (Phase 2), FastAPI AI service (Phase 3), and LangGraph agents + RAG (Phase 4). Phase 5 brings them together into a coherent production architecture. This topic covers the integration patterns, data contracts, and deployment configuration that make it all work as one product.

9.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Full Stack AI Architecture — How to Structure a Production AI Application	Theo (t3.gg)	YouTube → Search 'Full Stack AI Architecture Production Application Structure Theo t3gg'	Theo's architectural breakdown of a production AI SaaS — service boundaries, data contracts.
2	Video	Microservices vs Monolith for AI Applications — When to Split Services	Hussein Nasser	YouTube → Search 'Microservices vs Monolith AI Applications When to Split Hussein Nasser'	When to keep Python AI in the same service vs split it out. Critical architectural decision.
3	Video	Docker Compose for Full-Stack Development — Multi-Service Setup	Traversy Media	YouTube → Search 'Docker Compose Full-Stack Development Multi-Service Setup Traversy Media'	Run Next.js + Fastify + FastAPI + Qdrant + Redis + Postgres with one command.
4	Video	API Gateway Pattern — Route Requests Between Microservices	Hussein Nasser	YouTube → Search 'API Gateway Pattern Route Requests Microservices Hussein Nasser'	Fastify as API gateway — authentication, routing, rate limiting before forwarding to FastAPI.
5	Video	Environment Variable Management — 12-Factor App for AI Services	Theo (t3.gg)	YouTube → Search 'Environment Variable Management 12-Factor App AI Services Theo t3gg'	Manage secrets across 4 services — shared vs service-specific variables.
6	Video	End-to-End Testing for AI Applications — Playwright + MSW + Mock LLMs	Jack Herrington	YouTube → Search 'End to End Testing AI Applications Playwright MSW Mock LLMs Jack Herrington'	E2E tests for the full AI stack — mock LLM responses in tests without changing application code.

9.2 — The Production Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          USER LAYER                                  │
│  Browser / Mobile App  →  Next.js (Vercel Edge)                      │
│  Vercel AI SDK useChat / useObject / Generative UI                   │
└──────────────────────────┬──────────────────────────────────────────┘
                           │  HTTPS + SSE streaming
                           ▼
```

```
┌──────────────────────────────────────────────────────────────────────┐
│                FASTIFY API GATEWAY (Railway / Fly.io)                 │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  Auth (Clerk JWT) → Rate Limit (Redis) → Plan Gate           │    │
│  │  Model Router → Semantic Cache Check → Request Logging       │    │
│  └──────────────────────────────────────────────────────────────┘    │
│  Routes:                                                              │
│  /api/v1/chat     → proxies streaming to FastAPI                      │
│  /api/v1/documents→ uploads to S3, queues BullMQ job                 │
│  /api/v1/search   → proxies to FastAPI search                         │
│  /api/v1/billing  → Stripe API                                        │
└──────────────────────────────────────────────────────────────────────┘
       │  Internal HTTP (service mesh)
       ▼
┌──────────────────────────────────────────────────────────────────────┐
│              FastAPI AI SERVICE (Railway / Fly.io)                    │
│  LangChain LCEL chains + LlamaIndex indexes + LangGraph agents        │
│  RAGAS evals + LangSmith tracing + Semantic cache + Model router      │
└──────────────────────────────────────────────────────────────────────┘
       │            │            │            │
       ▼            ▼            ▼            ▼
   Qdrant DB      PostgreSQL     Redis        LLM APIs
   (vectors)      (Prisma)       (cache)      (OpenAI/
   (Railway)      (Railway)      (Railway)     Anthropic/
                                               Ollama)
       │
       ▼
   Prefect Pipelines (document ingestion, embedding, indexing)
   BullMQ Workers   (async doc processing, email, webhooks)
```

9.3 — Docker Compose for Local Development

```yaml
# docker-compose.yml — start everything with 'docker compose up'
version: '3.9'
services:
  nextjs:
    build: ./frontend
    ports: ['3000:3000']
    environment:
      - NEXT_PUBLIC_API_URL=http://fastify:4000

  fastify:
    build: ./backend
    ports: ['4000:4000']
    depends_on: [postgres, redis]
    environment:
      - DATABASE_URL=postgresql://postgres:dev@postgres:5432/aidb
      - REDIS_URL=redis://redis:6379
```

```
      - FASTAPI_URL=http://fastapi:8000

  fastapi:
    build: ./ai-service
    ports: ['8000:8000']
    depends_on: [qdrant, redis]
    environment:
      - QDRANT_URL=http://qdrant:6333
      - OPENAI_API_KEY=${OPENAI_API_KEY}

  qdrant:
    image: qdrant/qdrant:latest
    ports: ['6333:6333']
    volumes: ['qdrant_storage:/qdrant/storage']

  postgres:
    image: postgres:16-alpine
    environment: {POSTGRES_DB: aidb, POSTGRES_PASSWORD: dev}

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

volumes:
  qdrant_storage:

# Result: 6 services, one command, <90 seconds to start
# Developer experience that gets GitHub stars on your open source project
```

9.4 — Stop Conditions

- ☐ 'docker compose up' starts all 6 services in <90 seconds — all health checks pass
- ☐ End-to-end request trace: user chat message → Fastify → FastAPI → LangGraph → Qdrant → streamed response
- ☐ E2E Playwright test: full chat flow with mocked LLM — test passes without real API keys
- ☐ Shared auth: JWT from Clerk verified by both Fastify and FastAPI — single sign-on across services
- ☐ Shared Redis: Fastify rate limits + FastAPI semantic cache both use same Redis instance
- ☐ Service discovery: all services communicate by hostname, not IP — works in Docker, Railway, and Kubernetes

TOPIC 10 — AI Cost Analytics & Financial Dashboard · *Week 10 · Show stakeholders the ROI of your optimisations*

Every cost optimisation you implement in Topic 6 needs to be measurable and communicable. This topic covers building a cost analytics dashboard that tracks LLM spend, cache hit rates, model distribution, and cost per user — the metrics that matter to both engineering and business stakeholders.

10.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Recharts Tutorial — Build Analytics Dashboards in React	Lama Dev	YouTube → Search 'Recharts Tutorial Build Analytics Dashboards React Lama Dev'	Recharts for the cost analytics dashboard — line charts, bar charts, pie charts in React.
2	Video	Build a Real-Time Analytics Dashboard with Next.js and PostgreSQL	Traversy Media	YouTube → Search 'Real-Time Analytics Dashboard Next.js PostgreSQL Traversy Media'	Architecture for real-time metrics — polling vs WebSocket for dashboard updates.
3	Video	OpenAI Token Costs — How to Calculate and Track API Spending	David Ondrej	YouTube → Search 'OpenAI Token Costs Calculate Track API Spending David Ondrej'	Token cost calculation for each model — build this into your usage tracking middleware.
4	Video	PostHog for AI Applications — Track LLM Calls, Costs, and User Behaviour	PostHog (official channel)	YouTube → PostHog YouTube → Search 'PostHog AI Applications Track LLM Costs User Behaviour'	PostHog's new LLM analytics features — track AI calls alongside user behaviour.

10.2 — Dashboard Metrics to Track

Daily LLM Spend ($)	Total API cost per day, by provider (OpenAI / Anthropic / Google), by model tier
Cache Hit Rate (%)	% of requests served from semantic cache — direct measure of cost savings
Model Tier Distribution	% of requests routed to Tier 0 (cache) / Tier 1 (micro) / Tier 2 (standard) / Tier 3 (power)
Cost Per User	Average daily/monthly LLM cost per active user — tracks unit economics
P95 Latency by Tier	95th percentile response time per model tier — quality of service metrics
Eval Score Trend	RAGAS faithfulness + answer relevancy over time — quality regression detection
Token Usage by Feature	Tokens consumed by RAG search / agents / extraction / generation — find expensive features
Monthly Projection	Projected end-of-month cost based on current daily burn rate + growth rate

10.3 — Stop Conditions

☐ Cost tracking middleware: every LLM call logs model, tokens, cost, user_id, feature to PostgreSQL
☐ Dashboard: daily cost chart, cache hit rate gauge, model tier pie chart — all live data
☐ Cost alert: Slack notification when daily spend exceeds threshold (e.g., $50/day)
☐ Monthly report: automated PDF report showing cost trend, top features by cost, savings from caching
☐ ROI calculation: document: before caching $X/day → after caching $Y/day → savings Z% — stakeholder-ready

🗓 Phase 5 — Week-by-Week Schedule (16 Weeks)

Phase 5 alternates between framework learning and production system building. Every other week you are improving the capstone. By Week 16, the Research Agent is a complete, polished, production-grade product.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 1	LangChain LCEL — Basics + Composition	LCEL Official Tutorial Series — LangChain (all 4 parts) LCEL Deep Dive — James Briggs	Rebuild Phase 4 RAG chain using LCEL | operator Add RunnableParallel for concurrent retrieval	RAG chain uses LCEL; parallel retrieval working; streaming confirmed
Wk 2	LCEL Advanced + LlamaIndex Intro	Building Production RAG LCEL — Lance Martin LlamaIndex vs LangChain — Sam Witteveen LlamaIndex Full Course (first 1.5 hrs) — Sam Witteveen	Add .with_fallbacks() and .with_retry() to all chains Build first LlamaIndex VectorStoreIndex	Chain failover works; LlamaIndex index built and queryable
Wk 3	LlamaIndex Advanced + Vercel AI SDK Production	LlamaIndex Router Query Engine — Andrei Dumitrescu LlamaIndex Sub-Question Query Engine — Sam Witteveen Vercel AI SDK 3.4 New Features — Vercel official	Build Router Query Engine over 3 domain indexes Connect useChat to FastAPI backend	Router engine routes correctly; useChat streams from FastAPI
Wk 4	LLM Evaluation — Build the Eval System	LLM Evaluation Full Course — DeepLearning.AI (complete) RAGAS Complete Guide v0.2 — Sam Witteveen LLM as Judge — Sam Witteveen	Create 100-question golden test set Set up RAGAS evaluation pipeline Build LLM-as-judge scorer	RAGAS runs on golden test set; all 5 metrics computed and above thresholds
Wk 5	Continuous Eval + Prompt Versioning	Continuous Evaluation GitHub Actions — Hamel Husain LangSmith Prompt Hub — LangChain official AB Testing LLM Prompts — Sam Witteveen	RAGAS eval runs in GitHub Actions CI Migrate all prompts to LangSmith Prompt Hub	CI blocks PR with eval regression; prompts versioned in LangSmith
Wk 6	Model Routing + Semantic Caching	LLM Router Route Queries Right Model — Sam Witteveen Semantic Caching GPTCache — Zep AI Reducing OpenAI API Costs 10 Techniques — AI Explained	Build query complexity classifier Implement semantic cache with Redis + Qdrant Measure cost reduction	Router routes correctly; cache hit rate >30%; cost reduction documented
Wk 7	Multi-Modal AI — Vision + Audio	GPT-4o Vision Complete Tutorial — Dave Gray OpenAI Whisper Complete Guide — Tech With Tim Build Voice AI Assistant — Pixegami	Invoice image extraction working Voice transcription with Whisper End-to-end voice assistant prototype	Invoice OCR extracts all fields; voice assistant responds in <3 seconds
Wk 8	Multi-Modal RAG + AI UX Patterns	Multi-Modal RAG Images Charts Tables — James Briggs AI UX Design Patterns Users Trust — Lenny Podcast Streaming UI Build Great Experiences — Vercel official	Index PDF pages as images in Qdrant Implement 3 AI UX patterns: citations, streaming cursor, follow-ups	Multi-modal RAG working on chart data; all 3 UX patterns live in capstone
Wk 9	Full-Stack Integration	Full Stack AI Architecture — Theo t3gg Docker Compose Full-Stack Development — Traversy Media API Gateway Pattern — Hussein Nasser	'docker compose up' starts all 6 services E2E Playwright test for full chat flow	All services communicate; E2E test passes with mocked LLMs

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 10	Cost Analytics Dashboard	Recharts Tutorial Analytics Dashboards — Lama Dev OpenAI Token Costs Calculate Track — David Ondrej PostHog AI Applications — PostHog official	Cost tracking middleware for all LLM calls Dashboard: daily spend, cache rate, model distribution	Dashboard shows live data; Slack alert when spend exceeds threshold
Wk 11	Generative UI + Advanced Agent UI	Generative UI Vercel AI SDK RSC — Delba Vercel Multi-Step Tool Calling Vercel AI SDK — Vercel official Human-in-the-Loop UX — Design AI Conference	One Generative UI feature: query returns chart component HITL confirmation for agent email actions	Generative UI renders chart; HITL shows preview before action
Wk 12	Capstone — Core Build (Research Agent)	Review: LangGraph agent patterns Review: LangSmith tracing	Build AI Research Agent: - Web search (Tavily) - RAG knowledge base - Multi-step synthesis - HITL for sensitive actions	Agent completes 10-step research queries; all steps traced in LangSmith
Wk 13	Capstone — Full-Stack Wiring	Review: Vercel AI SDK + FastAPI streaming	Connect Research Agent to Next.js frontend Stream agent reasoning to UI All AI UX patterns applied	End-to-end: type research query → see agent thinking → streamed final report
Wk 14	Capstone — Eval + Cost Optimisation	Review: RAGAS + model routing	RAGAS eval on capstone — all metrics above threshold Model routing + semantic caching active Cost dashboard showing real data	RAGAS passing in CI; routing saves documented; dashboard shows live data
Wk 15	Capstone — Polish + Performance	Review: performance optimisation from Phase 1	Lighthouse >90 on all pages Skeleton screens for all AI loading states Mobile responsive AI chat UI	Lighthouse passes; no jank during streaming; works on mobile
Wk 16	Deploy + Open Source + README	Review: Railway + Docker deploy guides	Deploy all services to Railway Write README with: - Live demo link - Architecture diagram - Eval benchmark table - One-command setup	Live URL working; README gets first GitHub star; portfolio updated

⍰ Phase 5 Capstone — AI Research Agent (Full-Stack Showcase)

This is Portfolio Project #5 — the capstone that demonstrates your full-stack AI engineering capability. Every skill from all 5 phases comes together: React streaming UI, Fastify API gateway, FastAPI + LangGraph agent, multi-modal document processing, hybrid RAG, eval suite, cost dashboard, and production deployment. This is the project you walk into interviews with.

Full Project Specification

Project Name	ai-research-agent (GitHub repo name)
The Product	An AI research assistant that answers complex questions by searching the web, querying a knowledge base, analysing documents (including images/PDFs), and synthesising a cited report
Frontend	Next.js 14 App Router + Vercel AI SDK + Tailwind + Shadcn/UI — streaming reasoning visible
API Gateway	Fastify — auth (Clerk), rate limiting (Redis), model routing, semantic caching
AI Service	FastAPI + LangGraph multi-step agent + LangChain LCEL chains + LlamaIndex indexes
Agent Tools	Tavily web search, Qdrant RAG search, GPT-4o Vision (document analysis), Calculator, Code executor
Retrieval	Hybrid RAG (BM25 + dense + RRF) + HyDE + contextual retrieval + cross-encoder reranking
Evaluation	RAGAS automated eval in CI — all 5 metrics above thresholds on 100-question golden test set
Cost Optimisation	Model routing (4 tiers) + semantic caching — target <$0.01 per research query
Multi-modal	Upload PDF → agent extracts text + images → synthesises answer including chart data
Observability	LangSmith traces + cost analytics dashboard + eval score trend chart
Deployment	'docker compose up' local dev + Railway production + Vercel frontend

Required Features — Build All of These

Research Agent Capabilities

- Multi-step research: agent breaks complex query into sub-questions, researches each, synthesises
- Source diversity: web search (Tavily) + knowledge base (Qdrant) + uploaded documents
- Multi-modal: upload a PDF or image — agent extracts and incorporates the content
- Citations: every factual claim in the report links to its source (URL or document page)
- Depth control: user selects 'Quick Answer' (1 search) or 'Deep Research' (5–10 steps)

- HITL: agent shows planned research steps before executing — user can approve or modify

AI UX Features (all 10 patterns implemented)

- Progressive disclosure: short answer first, expandable detailed report below
- Streaming reasoning: user sees 'Searching web... Found 3 sources... Synthesising...' in real time
- Source citations: every claim has a numbered reference, clickable to source
- Confidence indicator: 'High / Medium / Low confidence' based on source quality
- Suggested follow-ups: 3 related questions generated after every research report
- Abort button: user can cancel long research tasks mid-execution
- Generative UI: data query returns a chart component, not text
- Feedback: thumbs up/down + 'Correct this answer' on every response

Production Quality Metrics (README must show these)

- RAGAS Faithfulness: >0.87 on 100-question golden test set
- RAGAS Answer Relevancy: >0.82
- Average research latency: <8 seconds for Quick Answer, <45 seconds for Deep Research
- Semantic cache hit rate: >35% after 1 week of usage
- Average cost per query: <$0.008 (with routing + caching active)
- Lighthouse performance: >90 on all pages

GitHub README Requirements (what gets stars)

- Live demo link at the top — no exceptions
- 30-second GIF showing the agent researching a complex question in real time
- Architecture diagram with all 6 services and data flow
- RAGAS benchmark table: all 5 metrics on the golden test set
- Cost comparison: cost per query before optimisation vs after
- 'docker compose up' works in <2 minutes — tested from a fresh clone
- Technical blog post section: 'How I built the eval pipeline' — 500 words minimum

Estimated Time Investment

Weeks 1–2 (LangChain LCEL + LlamaIndex)	~20 hrs — 10 hrs videos + 10 hrs rebuilding pipelines
Week 3 (Vercel AI SDK production)	~12 hrs — 6 hrs videos + 6 hrs integration
Weeks 4–5 (Evaluation + Prompt versioning)	~22 hrs — 10 hrs courses + 12 hrs building eval system
Week 6 (Model routing + Semantic caching)	~16 hrs — 6 hrs videos + 10 hrs implementation + measurement
Weeks 7–8 (Multi-modal + AI UX)	~20 hrs — 10 hrs videos + 10 hrs building features

Weeks 9–10 (Integration + Cost analytics)	~18 hrs — 6 hrs videos + 12 hrs Docker + dashboard
Weeks 11–12 (Capstone — Generative UI + Agent)	~20 hrs — all building
Weeks 13–14 (Capstone — Full-stack + Evals)	~18 hrs — integration, testing, eval pipeline
Weeks 15–16 (Capstone — Polish + Deploy)	~14 hrs — polish, deploy, README, open source
Total Phase 5	~160 hours over 16 weeks ≈ 2.5 hrs/weekday + 5 hrs/weekend

Phase 5 Completion Check: You have completed Phase 5 when: (1) The AI Research Agent is live with a demo URL, (2) RAGAS scores are documented in the README, (3) Cost dashboard shows real savings from caching and routing, (4) 'docker compose up' works in under 2 minutes from a fresh clone, (5) The GitHub README has the live demo GIF, architecture diagram, and benchmark table. At this point you have a portfolio that demonstrates senior AI engineering capability.

Phase 5 trap: spending 12 weeks on framework comparisons (LangChain vs LlamaIndex vs plain Python) and never shipping. The framework wars are irrelevant to users. Your capstone uses both LangChain and LlamaIndex for what each does best — and plain Python where neither adds value. Ship the product.